

Assignment 1: Mini Banking Transaction Server

Due date: _____1405/07/29_____ Submission: _____LMS and GitHub_____

1. Overview

In this assignment you will build a small client–server banking system. Everything in it is simulated: the accounts, balances and customers are made up, so please do not put any real banking information in your code, test data or screenshots.

The goal is to get hands-on experience with several topics from the course: how a client and a server communicate, what happens when many clients connect at once, how shared state is kept consistent, and how a server should behave when it receives requests it cannot process. Error handling and logging are part of the grade.

2. Server Requirements

The server keeps a few simulated accounts, each with an initial balance, and must support at least the following operations:

- Balance inquiry
- Deposit
- Withdraw

A request could look like this:

```
{
  "action": "withdraw",
  "account": "A1001",
  "amount": 200
}
```

and a successful response like this:

```
{
  "status": "success",
  "account": "A1001",
  "balance": 800
}
```

You are free to design your own message format, as long as it is consistent and explained in your report. For communication you may choose any one of: HTTP, TCP sockets, or WebSocket.

2.1 Concurrency and shared state

The server must:

- Serve multiple clients concurrently.
- Let the maximum number of simultaneous clients be configured (through a config file, an environment variable or a command-line argument; it should not be hard-coded).
- Keep account information as shared state that every client handler works on.
- Handle simultaneous operations on the same account correctly. For example, if two withdrawals arrive on the same account at the same moment, the final balance must be one that could also be reached by running them one after the other.
- Never produce invalid results (such as a negative balance or a lost update) because of concurrent requests.

2.2 Transactions and ACID properties

Every operation that changes a balance is a transaction, and the server has to respect all four ACID properties. For this assignment that means:

Property	What it means here
Atomicity	An operation either completes fully or has no effect at all. If something goes wrong halfway (invalid input, insufficient funds, an exception, the server being killed), the balance must stay exactly as it was. If you implement a transfer between two accounts, the debit and the credit must happen together or not at all.
Consistency	Each transaction takes the system from one valid state to another. Balances never become negative, amounts are always positive numbers, and the total money in the system changes only through deposits and withdrawals.
Isolation	Concurrent transactions must not see each other's unfinished work. Running them in parallel must give the same result as running them one by one in some order (serializability). Lost updates and dirty reads are not acceptable.
Durability	Once the client has received `success`, the change must survive a server crash or restart. Keeping balances only in memory is therefore not enough: write the committed state to disk (a file, a write-ahead log, SQLite or any other database) before sending the reply.

How you achieve this is up to you (locks, the built-in transactions of a database, a single-writer queue, and so on), but you should be able to explain why your approach guarantees each of the four properties. In the demo, be ready to show at least:

- many concurrent withdrawals on the same account, ending with a correct final balance;
- a request that fails halfway, leaving the account unchanged;
- the server being killed and restarted, with all previously confirmed transactions still there.

2.3 Error handling

Invalid requests must be answered with a meaningful error, and the server must keep running. At minimum, the following cases have to be handled:

Case	Example
Unknown account	a request for account `Z9999` which does not exist
Insufficient funds	withdrawing 5000 from an account holding 800
Negative or invalid amount	`-50`, `0`, `"abc"`
Unsupported action	`"transfer"` when it is not implemented
Missing fields	a withdraw request without `amount`
Malformed request	broken JSON or random bytes on the socket

2.4 Logging

Every request and its response must be logged. Each log entry should include at least:

- Request ID
- Client IP
- Action
- Account ID
- Timestamp
- Result
- Latency

Plain text or JSON lines are both fine. A line might look like:

```
2026-10-12T10:41:07.512Z  req=7f3a  ip=192.168.56.11  action=withdraw
  account=A1001  result=success  balance=800  latency=3ms
```

3. Deployment

The client and the server must run on two separate virtual machines that are connected over a network (for example a host-only or bridged network in VirtualBox or VMware). Running both on the same machine is not accepted for this assignment.

The client has to connect to the real IP address of the server VM, not to localhost.

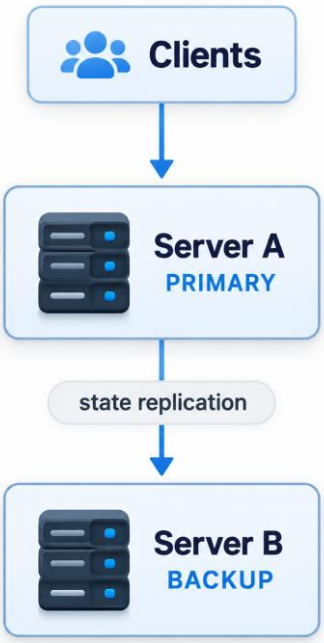
4. What to Submit

- Complete source code of the client and the server.
- A dependency file with library versions (e.g. `requirements.txt`, `package.json`, `go.mod`, `pom.xml`).
- A report of at most five A4 pages.
- A short screen-recorded video (up to 5 minutes) of the client and server running on their separate VMs. It must show: normal operations (balance inquiry, deposit, withdraw), several clients working at the same time, each of the error cases listed above, the log output, and the ACID demonstrations described in section 2.2.

The report should briefly cover: the communication method you chose, how concurrent clients are handled, how shared state is managed, how concurrent operations on the same account are kept correct, how each of the ACID properties is guaranteed, and how to run your code.

5. Advanced Bonus: Fault-Tolerant Banking Cluster (up to +10%)

For extra credit, extend the system into a fault-tolerant setup with at least two banking server instances arranged as primary and backup:



This part needs at least three VMs: two VMs for the banking servers (Server A and Server B) and at least one VM for the clients. Using more than one client VM is welcome.

Here is the situation. At the start, Server A is serving the clients and Server B is running next to it. While clients are sending deposits and withdrawals, Server A suddenly crashes: its process is killed or its VM is shut down. Some requests may have been halfway through at that moment, and some clients may have sent a request without ever getting a reply, so they don't know whether it was executed or not. Without any fault tolerance, the bank simply stops working, and any data that existed only on Server A is gone.

Your task is to design your own solution so that, in this situation, the service continues on Server B without anyone stepping in by hand, the account balances stay correct, and no transaction is lost or applied twice. Implement it and in your report, explain how your design works and why it handles this scenario correctly.

In the demo, Server A will be stopped on purpose while clients are still sending requests, and Server B has to take over and keep serving them.

Your submitted video must also show this scenario from start to end: clients sending requests, Server A being stopped while they are active, Server B taking over, and the account balances at the end.

6. Extreme Bonus: Network Partition and Split Brain (+15%)

This part is about network partitions. The two servers run on separate VMs, and at some point the network link between them breaks while both machines keep running. From that moment, neither server can reach the other, and neither can tell whether the other one has crashed or is simply unreachable. Clients may still be able to reach one side, the other, or both. If each server decides on its own to act as the primary and keeps accepting deposits and withdrawals, the account data on the two sides will drift apart, and when the network comes back there will be two conflicting versions of the balances. This situation is known as split brain.

Two separate VMs are required for this part, since the partition has to be a real network failure between machines, not a stopped process.

Your task is to design your own solution to this problem, implement it, and demonstrate it. During the demo, the network between the VMs will be cut while clients are active; we will watch how your system behaves during the split, then restore the connection and see how it recovers. In your report, explain the approach you chose and why it keeps the account data correct in this scenario.

Your submitted video must also show this scenario from start to end: the network between the server VMs being cut while clients are active, how your system behaves during the split, and how it recovers after the connection is restored.